Machine Learning

Lecture 3 - Linear Regression

Linear Regression

- Task: predict scalar-valued targets (e.g. stock
- prices) Architecture: linear function of the inputs

A Modular Approach to ML

- choose a model describing relationships between variables
- define a loss function quantifying how well the model fits the data
- choose a regularizer expressing preference over different
- models

  fit a model that minimizes the loss function and satisfies the regularizer's constraint/penalty, possibly using an optimization algorithm

Supervised Learning Setup

- Input $\mathbf{x} \in X$ (a vector of features)
- Target $t \in T$
- Data $D = \{(\mathbf{x}^{(i)}, t^{(i)}) \text{ for } i = 1, 2, ..., N\}$
- Objective: learn a function $f : X \rightarrow T$ based on the data such that $t \approx y = f(\mathbf{x})$

Simple Linear Regression

- The objective is to study the relationship between two variables X and Y.
- Regression analysis is the process of estimating a functional relationship between X and Y. A regression equation is often used to predict a value of Y for a given value of X.

Simple Linear Regression Model

$$Y = \beta_0 + \beta_1 X$$

where

y = dependent variable

x = independent variable

β_0= y-intercept

β_1= slope of the line

Simple Linear Model

$$Y = \beta_0 + \beta_1 X$$

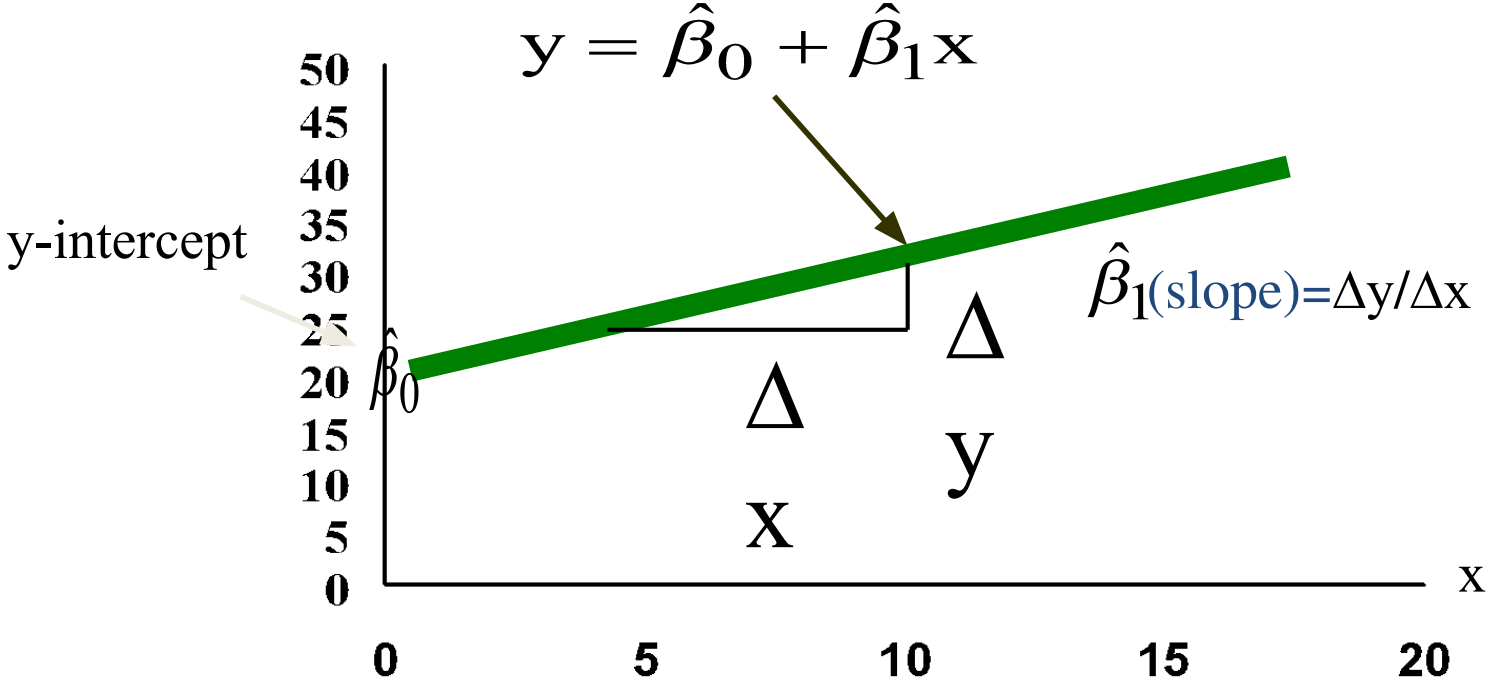
This model is

Simple: only one X

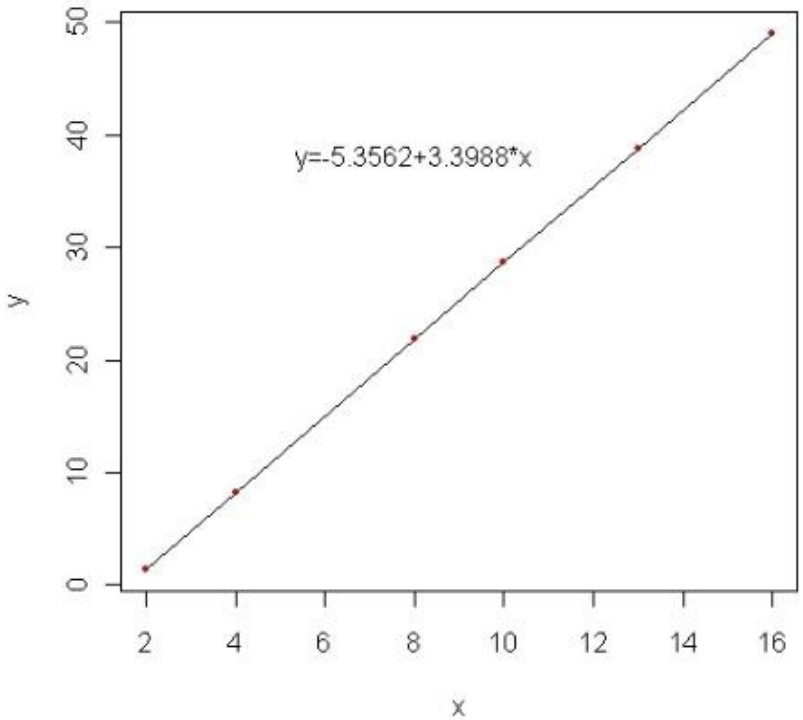
Linear in the parameters: No parameter appears as exponent or is multiplied or divided by another parameter

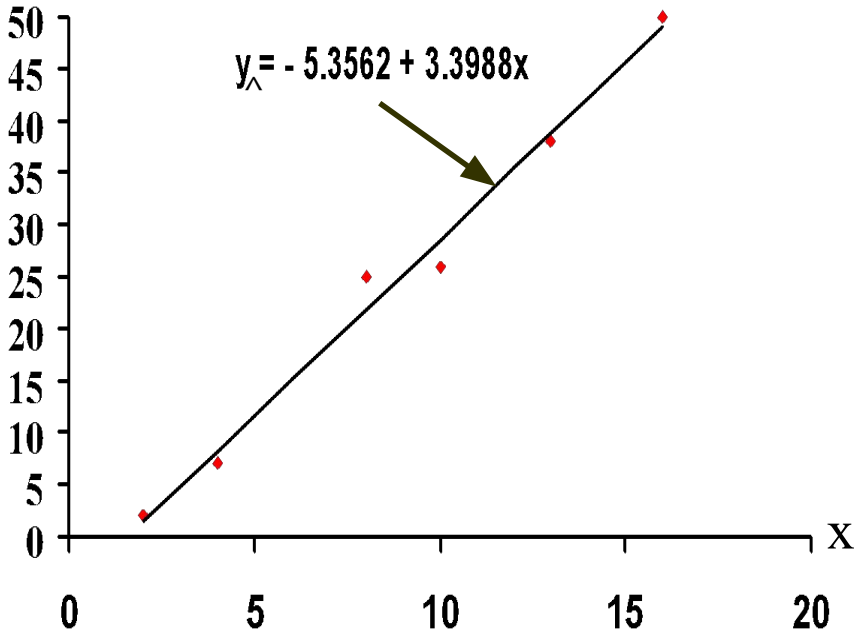
Linear in the predictor variable (X): X appears only in the first power.

Deterministic Component of Model



Mathematical vs Statistical Relation





Error

The scatterplot shows that the points are not on a line, and so, in addition to the relationship: $Y = \beta_0 + \beta_1 X$

The Y's are the response (or dependent) variable. The x's are the independent variables

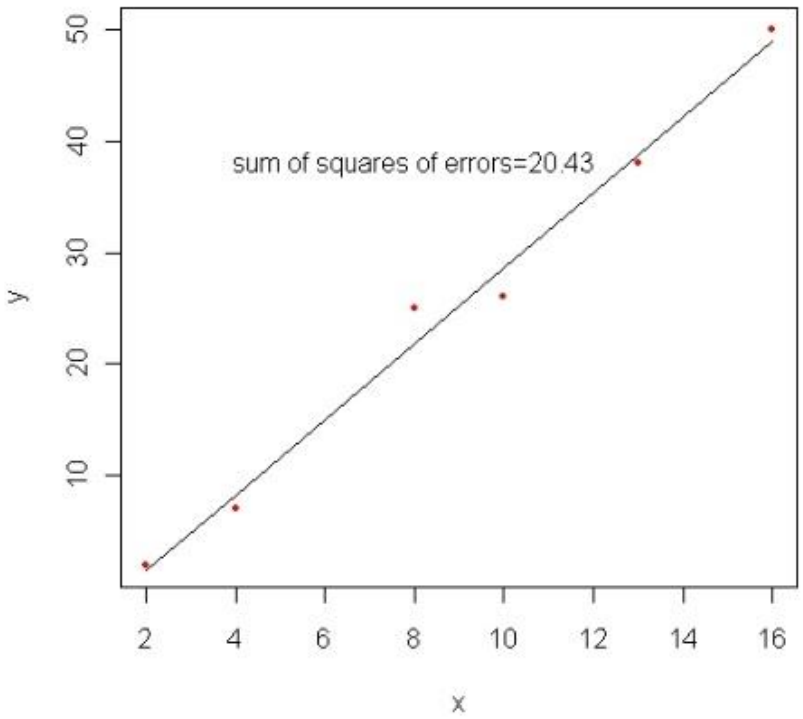
Least Squares:

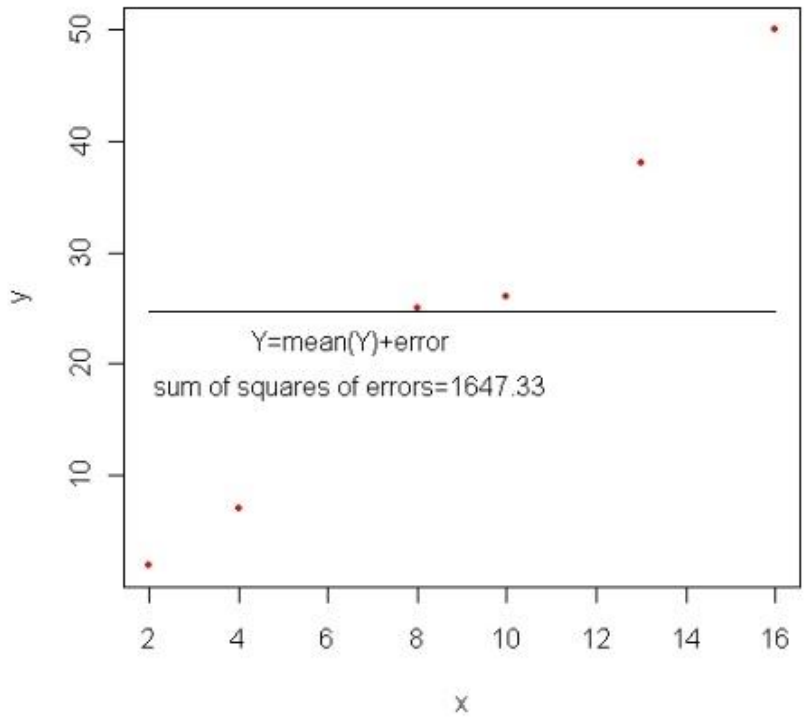
Minimize $$\sum_{i=1}^{n} \varepsilon_i^2 = \sum_{i=1}^{n} (y_i - \beta_0 - \beta_i x_i)^2$$

$$\Longrightarrow \quad \hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

The quantities $R_i = y_i - \hat{y}_i$ are called the residuals. If we assume a normal error, they should look normal.

Minimizing error





The Least Squares Regression Line where

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

$$\hat{\beta}_1 = \frac{SS_{xy}}{SS_x} \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

$$SS_x = \sum (x_i - \bar{x})^2 = \sum x_i^2 - \frac{\left(\sum x_i\right)^2}{n}$$

$$SS_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \sum x_i y_i - \frac{\left(\sum x_i\right)\left(\sum y_i\right)}{n}$$

Example

An educational economist wants to establish the relationship between an individual's income and education. He takes a random sample of 10 individuals and asks for their income (in $1000s) and education (in years). The results are shown below. Find the least squares regression line.

Education	11	12	11	15	8	10	11	12	17	11
Income	25	33	22	41	18	28	32	24	53	26

Dependent and Independent Variables

The dependent variable is the one that we want to forecast or analyze.
The independent variable is hypothesized to affect the dependent variable.
In this example, we wish to analyze income and we choose the variable individual's education that most affects income. Hence, y is income and x is individual's education

First Step:

$$\sum x_i = 118$$

$$\sum x_i^2 = 1450$$

$$\sum y_i = 302$$

$$\sum y_i^2 = 10072$$

$$\sum x_i y_i = 3779$$

Sum of Squares:

$$SS_{xy} = \sum x_i y_i - \frac{(\sum x_i)(\sum y_i)}{n} = 3779 - \frac{(118)(302)}{10} = 215.4$$

$$SS_x = \sum x_i^2 - \frac{(\sum x_i)^2}{n} = 1450 - \frac{(118)^2}{10} = 57.6$$

Therefore, $\hat{\beta}_1 = \frac{SS_{xy}}{SS_x} = \frac{215.4}{57.6} = 3.74$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = \frac{302}{10} - 3.74\frac{118}{10} = -13.93$$

The Least Squares Regression Line

The least squares regression line is

$$\hat{y} = -13.93 + 3.74x$$

Interpretation of coefficients:

*The sample slope $\hat{\beta}_1 = 3.74$ tells us that on average for each additional year of education, an individual's income rises by $3.74 thousand.

The y-intercept is $\hat{\beta}_0 = -13.93$. This value is the expected (or average) income for an individual who has 0 education level (which is meaningless here)

Model

Model: a *linear* function of the features $\mathbf{x} = (x_1, \ldots, x_D) \in \mathrm{R}^D$ to make prediction $y \in \mathrm{R}$ of the target $t \in \mathrm{R}$:

$$y = f(\mathbf{x}) = \sum_j w_j x_j + b = \mathbf{w}\,\mathbf{x}^\top + b$$

- Parameters are weights $\mathbf{w}$ and the bias/intercept b
- Want the prediction to be close to the target: $y \approx t$.
- Highly interpretable model, useful for debugging.

Loss Function

Loss function $L(y, t)$ defines how badly the algorithm's prediction y fits the target t for some example $\mathbf{x}$.

Squared error loss function: $L(y, t) = \frac{1}{2}(y - t)^2$

- $y - t$ is the residual, and we want to minimize this magnitude
- $\frac{1}{2}$ makes calculations convenient.

Cost function: loss function averaged over all training examples also called *empirical* or *average loss*.

$$\mathcal{J}(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \left(y^{(i)} - t^{(i)} \right)^2 = \frac{1}{2N} \sum_{i=1}^{N} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - t^{(i)} \right)^2$$

Vectorization

Loops v.s. Vectorized Code

- We can compute prediction for one data point using a for loop:

```
y = b
for j in range(M):
        y += w[j] * x[j]
```

- But, excessive super/sub scripts are hard to work with, and Python loops are slow.
- Instead, we express algorithms using vectors and matrices.

$$\mathbf{w} = (w_1, \ldots, w_D)^\top \qquad \mathbf{x} = (x_1, \ldots, x_D)^\top$$

$$y = \mathbf{w}^\top \mathbf{x} + b$$

- This is simpler and executes much faster:

```
y = np.dot(w, x) + b
```

Benefits of Vectorization

Why vectorize?

- The code is simpler and more readable. No more dummy variables/indices!
- Vectorized code is much faster
  - Cut down on Python interpreter overhead
  - Use highly optimized linear algebra libraries (hardware support)
  - Matrix multiplication very fast on GPU

Predictions for the Dataset

- Put training examples into a design matrix **X**.
- Put targets into the target vector **t**.
- We can compute the predictions for the whole dataset.

$$\begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_D^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_D^{(2)} \\ \vdots & \vdots & & \vdots \\ x_1^{(N)} & x_2^{(N)} & \dots & x_D^{(N)} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_D \end{pmatrix} + b \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{pmatrix}$$

Computing Squared Error Cost

We can compute the squared error cost across the whole dataset.

$$\mathcal{J} = \frac{1}{2N}\|\mathbf{y} - \mathbf{t}\|^2$$

Sometimes we may use $J = \frac{1}{2}\|\mathbf{y} - \mathbf{t}\|^2$, without a normalizer. This would correspond to the sum of losses, and not the averaged loss. The minimizer does not depend on N (but optimization might!).

Combining Bias and Weights

We can combine the bias and the weights and add a column of 1's to design matrix.

Our predictions become

$$\mathbf{y} = \mathbf{X}\mathbf{w}.$$

$$\mathbf{X} = \begin{bmatrix} 1 & [\mathbf{x}^{(1)}]^\top \\ 1 & [\mathbf{x}^{(2)}]^\top \\ 1 & \vdots \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \quad \text{and} \quad \mathbf{w} = \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \end{bmatrix} \in \mathbb{R}^{D+1}$$

Example

Python has methods for finding a relationship between data-points and to draw a line of linear regression. We will show you how to use these methods instead of going through the mathematic formula.

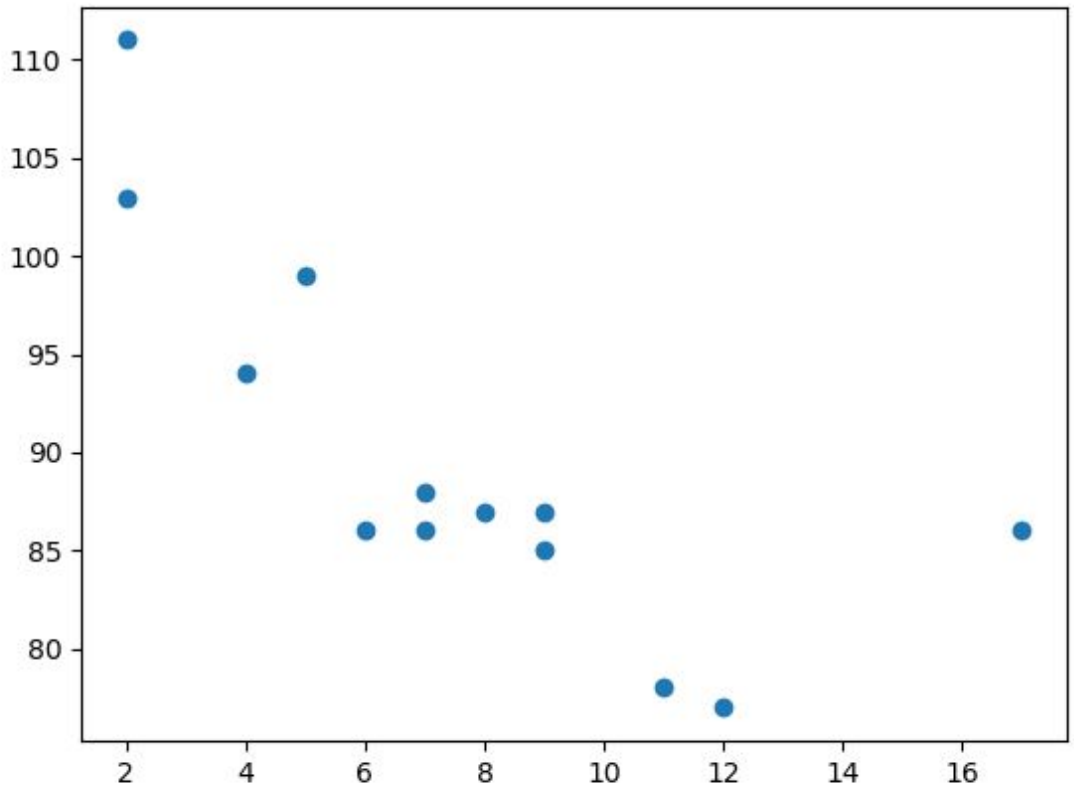
In the example below, the x-axis represents age, and the y-axis represents speed. We have registered the age and speed of 13 cars as they were passing a tollbooth. Let us see if the data we collected could be used in a linear regression:

Example

```
import matplotlib.pyplot as plt

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

plt.scatter(x, y)
plt.show()
```



Example

```
import matplotlib.pyplot as plt
from scipy import stats

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

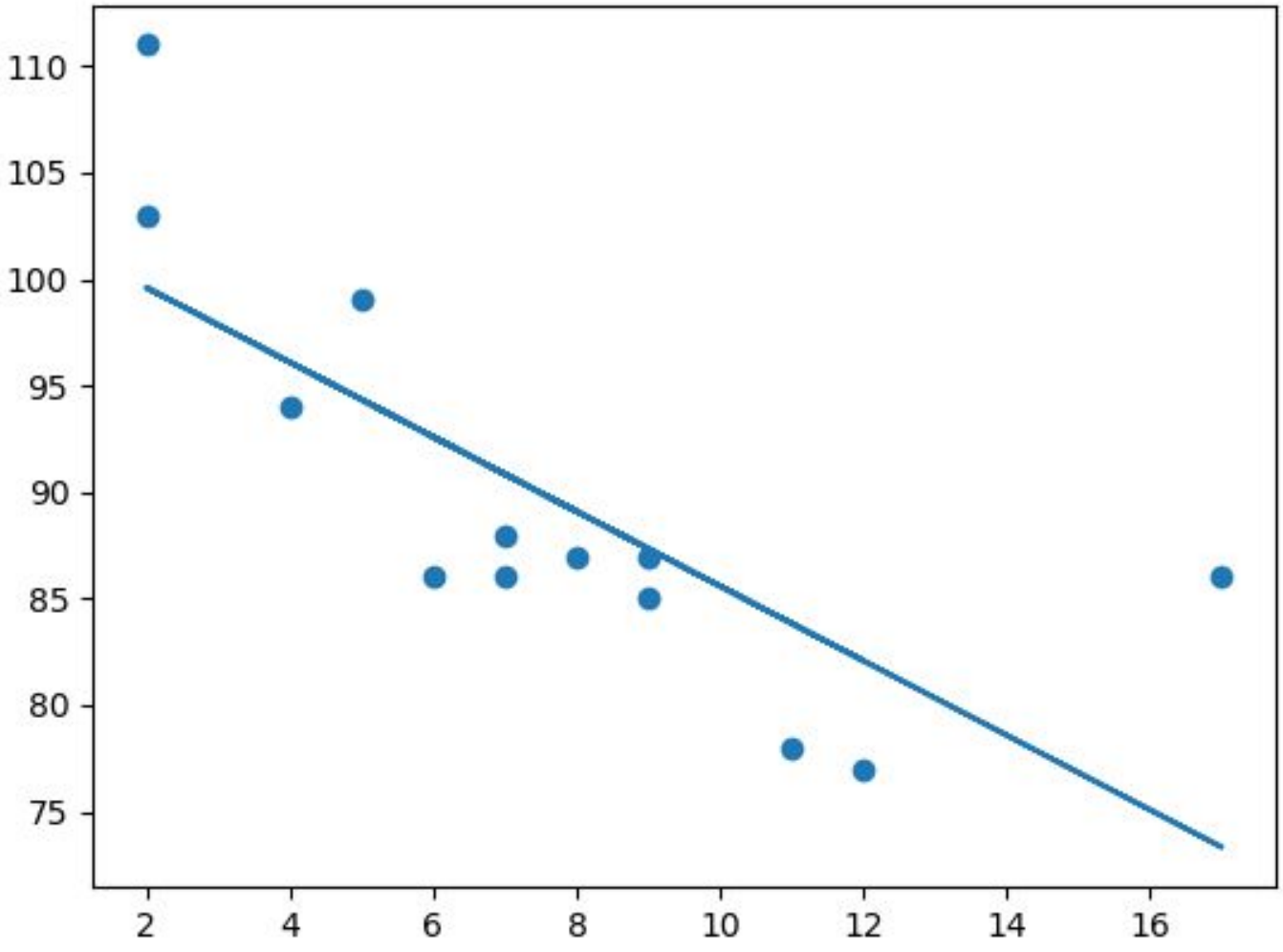
slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x):
  return slope * x + intercept

mymodel = list(map(myfunc, x))

plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
print("loss:", std_err)
```

Example



Loss: 0.428

Example 2: Linear Regression in Public Policy

A government wants to understand how spending on employment programs affects the unemployment rate.
We will build a model to predict unemployment.

Government Spending (Million $)	Unemployment Rate (%)
50	9.5
60	9
70	8.7
80	8.0
90	7.8
100	7.2
110	6.8
120	6.5

Example 2: Linear Regression in Public Policy

```
import numpy as np
from sklearn.linear_model import LinearRegression

spending = np.array([50, 60, 70, 80, 90, 100, 110, 120]).reshape(-1, 1)
unemployment = np.array([9.5, 9.0, 8.7, 8.0, 7.8, 7.2, 6.8, 6.5])
# Create model
model = LinearRegression()
# Train model
model.fit(spending, unemployment)
# Predict unemployment for new spending value
predicted_rate = model.predict([[95]])

print("Predicted unemployment rate:", predicted_rate[0])

# Model parameters
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```

Quiz:

Look at the following training set for a decision tree that classifies names as either location or person names:

name	attribute 1 first name	attribute 2 geo-term	label label
George Washington Bridge	yes	yes	location
John Hancock Tower	yes	yes	location
Ronald Reagan Building	yes	yes	location
Azore Islands	no	yes	location
Michael Jackson	yes	no	person
Justin Bieber	yes	no	person
Lady Gaga	no	no	person
Adele	yes	no	person
Stalin	no	no	person

Which attribute is more useful, attribute 1 or attribute 2? Why?